

Exercise 2

May 26, 2026

1 Exercise 2

Phase 2 – State and behavior (Week 2)

Objects exist because state persists.

Exercise 2: Oscilloscope simulator

```
[1]: import numpy as np
import matplotlib.pyplot as plt
```

Create:

```
class Oscilloscope:
```

Internal state:

```
sample_rate
voltage_scale
time_scale
current_signal
```

Methods:

```
load_signal()
autoscale()
measure_rms()
display()
```

```
[2]: class Oscilloscope:

    def __init__(self, fs, voltage_scale=[-1, 1], time_scale=[-0.1, 0.1],
                  current_signal=None):
        self.fs = fs
        self.voltage_scale = voltage_scale
        self.time_scale = time_scale
        self.time = np.arange(int(fs*time_scale[0]), int(fs*time_scale[1]))/
        self.fs
        if current_signal is not None:
            self.current_signal_orig = np.array(current_signal)
            self.current_signal = self.adj_signal_size(self.current_signal_orig)
        else:
```

```

        self.current_signal_orig = None
        self.current_signal = None

def adj_signal_size(self, signal):
    """
    Helper function to expand/contract signal to length of self.time.
    Expansion is done by periodic repetition.
    """
    if signal.size > self.time.size:
        signal = signal[:self.time.size]
    else:
        while signal.size < self.time.size:
            signal = np.hstack((signal, signal))
        signal = signal[:self.time.size]
    ix = np.where(self.time<0)[0]
    # periodic signal starting at t=0, tail of signal becomes
    # signal for negative t
    signal = np.hstack((signal[len(ix):], signal[:len(ix)]))
    return signal

def load_signal(self, signal):
    self.current_signal_orig = np.array(signal)
    self.current_signal = self.adj_signal_size(self.current_signal_orig)

def autoscale(self):
    if self.current_signal_orig is not None:
        self.current_signal = self.current_signal_orig
        self.time = np.arange(self.current_signal.size)/self.fs
        aux = np.hstack((np.real(self.current_signal), np.imag(self.
↪current_signal)))
        self.voltage_scale = [np.min(aux), np.max(aux)]
        self.time_scale = [0, self.time[-1]]

def measure_rms(self):
    rms = np.sqrt(np.mean(np.abs(self.current_signal)**2))
    return rms

def display(self):

    if self.current_signal is None:
        raise ValueError("Signal x(t) is not defined")
    fig = plt.figure()
    ax1, ax2 = fig.subplots(2, 1, sharex=True)
    ax1.plot(self.time, np.real(self.current_signal), '-b')
    ax1.set_ylabel('Re[x(t)]')
    ax1.set_ylim(self.voltage_scale)
    ax1.grid(alpha=0.6)

```

```

        ax1.set_title(f'Signal  $x(t)$ ,  $f_s$ ={self.fs:.1f} Hz,  $V_{rms}$ ={self.
↪measure_rms()} V')
        ax2.plot(self.time, np.imag(self.current_signal), '-r')
        ax2.set_ylabel('Im[x(t)]')
        ax2.set_xlabel('t [sec]')
        ax2.set_ylim(self.voltage_scale)
        ax2.grid(alpha=0.6)

plt.show()

```

```

[3]: class Signal:
    """
    Pseudo continuous-time  $x(t)$  signal object with time axis  $tt$ .
     $x(t)$  is assumed to be sampled uniformly with sampling rate  $fs$ .
    """

    def __init__(self, tt, xt):
        if len(tt) != len(xt):
            raise ValueError("tt and xt must have the same length")
        self.tt = np.array(tt)
        dt = np.diff(self.tt)
        if not np.allclose(dt, dt[0]):
            raise ValueError("time samples must be uniformly spaced")
        self.xt = np.array(xt)
        self.fs = (self.tt.size-1)/(self.tt[-1]-self.tt[0])

    def duration(self):
        return self.tt.size/self.fs

    def energy(self):
        return np.sum(np.abs(self.xt)**2)/self.fs

    def plot(self):

        fig = plt.figure()
        ax1, ax2 = fig.subplots(2, 1, sharex=True)
        ax1.plot(self.tt, np.real(self.xt), '-b')
        ax1.set_ylabel('Re[x(t)]')
        ax1.grid(alpha=0.6)
        ax1.set_title(f'Signal  $x(t)$ ,  $f_s$ ={self.fs:.1f} Hz')
        ax2.plot(self.tt, np.imag(self.xt), '-r')
        ax2.set_ylabel('Im[x(t)]')
        ax2.set_xlabel('t [sec]')
        ax2.grid(alpha=0.6)

        plt.show()

```

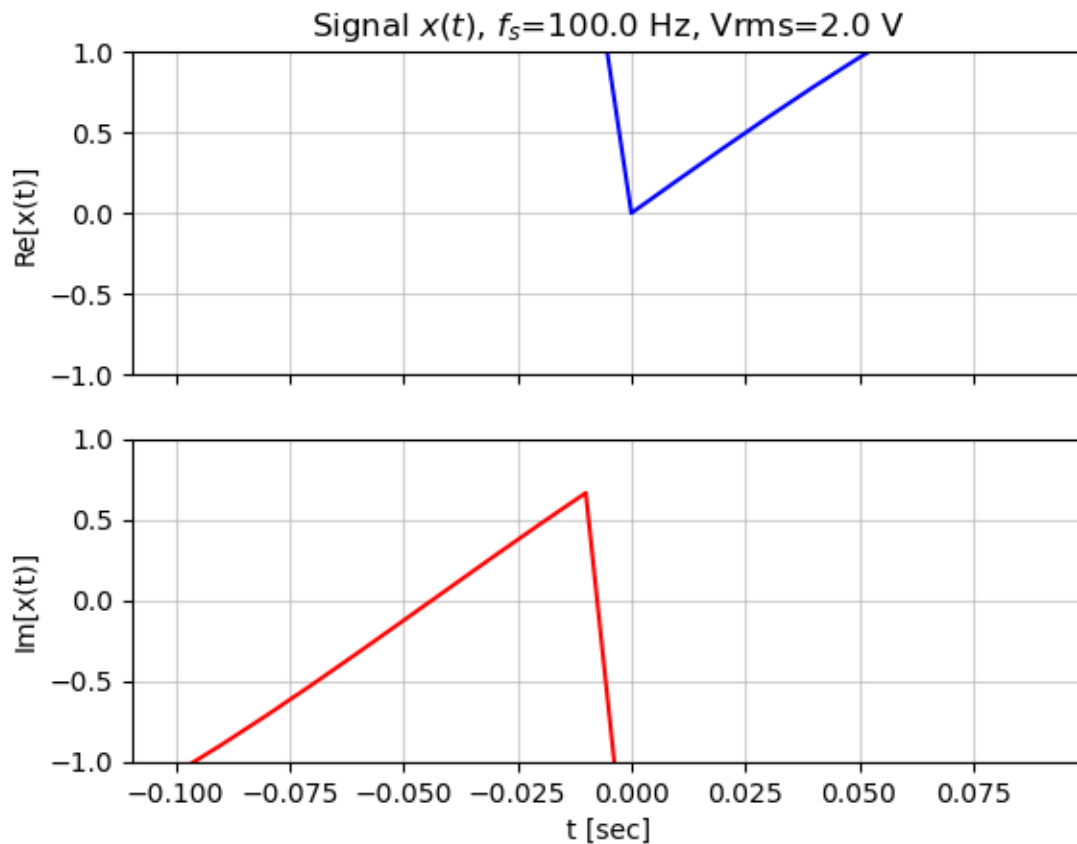
```
[4]: fs1 = 100
      tlen1 = 1
      tt1 = np.arange(int(np.round(tlen1*fs1)))/fs1
      A1, f1, th1 = 2, 1.6, -90
      x1t = Signal(tt1, A1*np.exp(1j*(2*np.pi*f1*tt1+np.pi/180*th1)))
```

```
[5]: scope1 = Oscilloscope(x1t.fs, current_signal=x1t.xt)
```

```
[6]: print(scope1.voltage_scale)
      print(scope1.time.size)
      print(scope1.measure_rms())
```

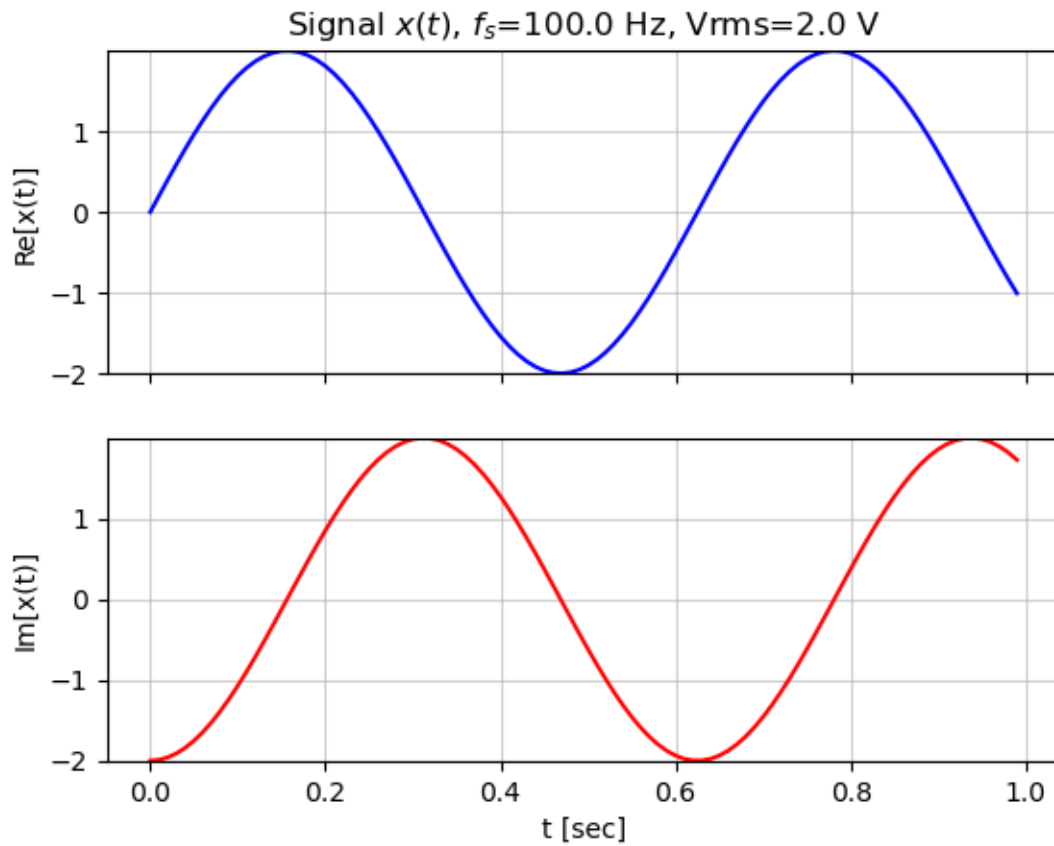
```
[-1, 1]
20
2.0
```

```
[7]: scope1.display()
```



```
[8]: scope1.autoscale()
```

```
[9]: scope1.display()
```

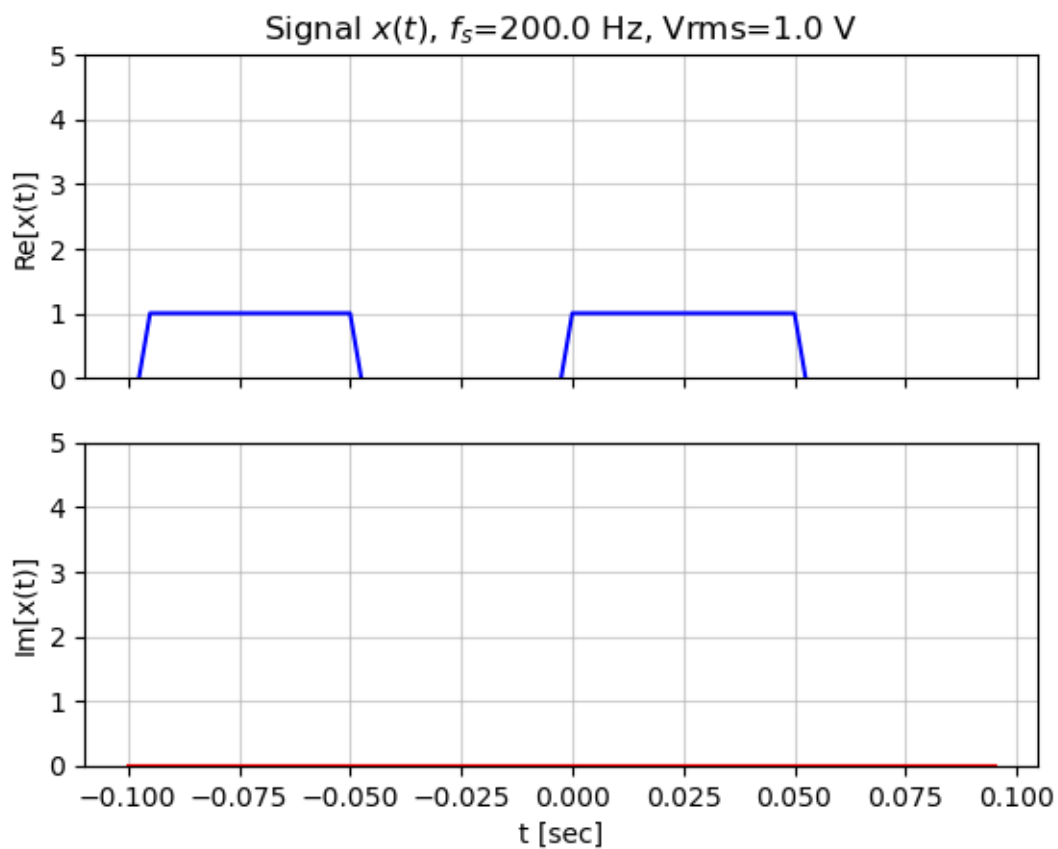


```
[10]: fs2 = 200
      tlen2 = 0.5
      tt2 = np.arange(int(np.round(tlen2*fs2)))/fs2
      A2, f2, th2 = 1, 10, -90
      x2t = Signal(tt2, A2*np.sign(np.cos(2*np.pi*f2*tt2+np.pi/180*th2)))
```

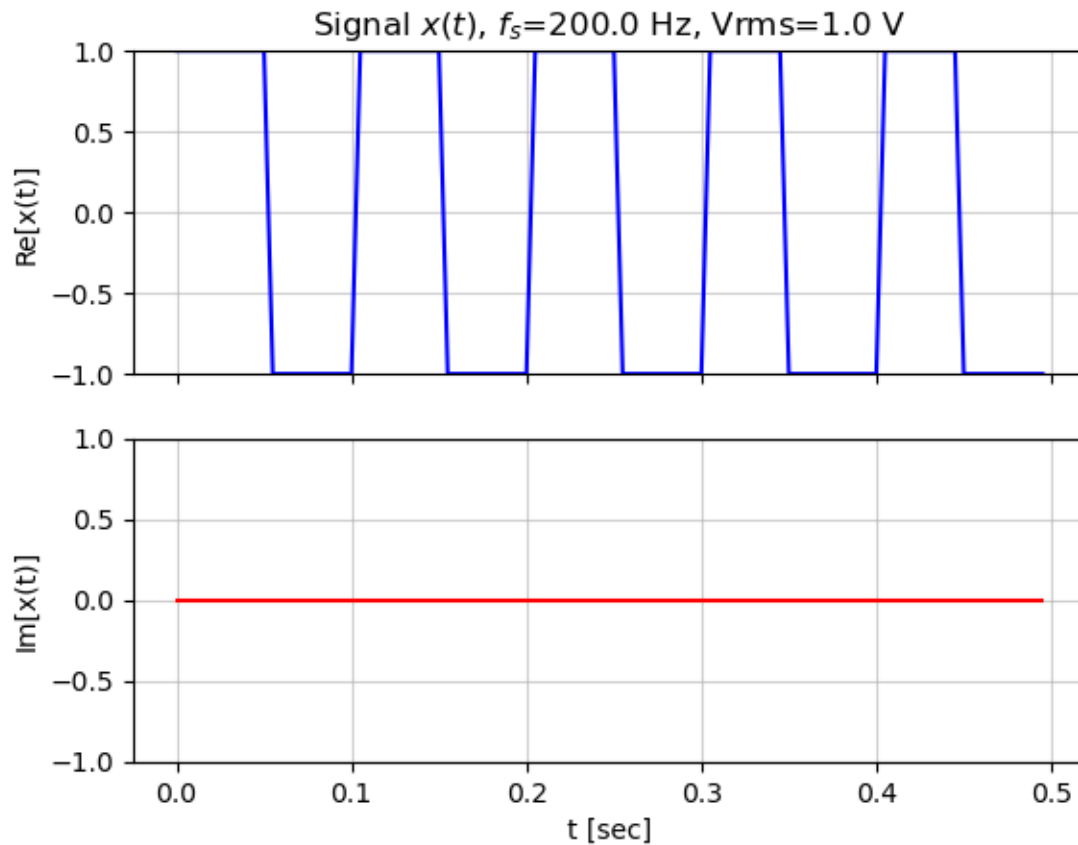
```
[11]: scope2 = Oscilloscope(x2t.fs, voltage_scale=[0, 5], current_signal=x2t.xt)
      print(scope2.voltage_scale)
      print(scope2.time.size)
      print(scope2.measure_rms())
```

```
[0, 5]
40
1.0
```

```
[12]: scope2.display()
```



```
[13]: scope2.autoscale()  
      scope2.display()
```



Notebook questions:

1. Which variables belong to object state?
2. Which should be method arguments?
3. What happens if two scopes observe different signals?

Answers:

1. `fs`, `voltage_scale`, `time_scale`, `current_signal_orig`
2. `current_signal`
3. They have different states and different method arguments

[]: